

DMP-EGNN 模型架構完整分析報告

Directed Message-Passing Equivariant Graph Neural Network

—— 特徵工程、模型計算流程與多模態融合的逐步剖析，並提出架構與演算法優化建議

專案位置	/home/vincent/drug_properties/fusion_model/
核心模型檔案	core/edmpnn_model_new.py (類別名稱 AEGNNM，於 models.py 中別名為 DMPEGNN)
特徵工程檔案	core/dmpegnn_data_utils.py, core/dmpegnn_dataset.py
融合整合檔案	core/models.py, core/model_factory.py, core/train_utils.py
任務類型	TDC ADMET 基準資料集之藥物性質預測 (分類：ROC-AUC；迴歸：MAE)
報告產出日期	2026-07-08
分析方法	逐行原始碼追溯 (三組平行程式碼探索代理，覆蓋約 4,600 行程式碼)，所有結論均附 file:line 引用

目錄

0. 執行摘要與模型定位	
1. 整體資料流程總覽	
2. 第一部分：資料前處理與特徵工程	
3. 第二部分：DMP-EGNN 核心模型架構	
4. 第三部分：多模態融合與訓練流程	
5. 已知程式碼問題彙整表	
6. 架構與演算法優化建議（依優先級排序）	
7. 結論	

0. 執行摘要與模型定位

DMP-EGNN（程式碼內部類別名為 `AEGNNM`，於 `models.py:10` 匯入時別名為 `DMPEGNN`，需與同檔案匯入的舊版 `aegnnm_model.AEGNNM`（別名 `AEGNN`，無 `dmp_steps / geo_gate`）區分）是 `fusion_model` 專案中最主要的 3D 感知圖神經網路編碼器，結合了：

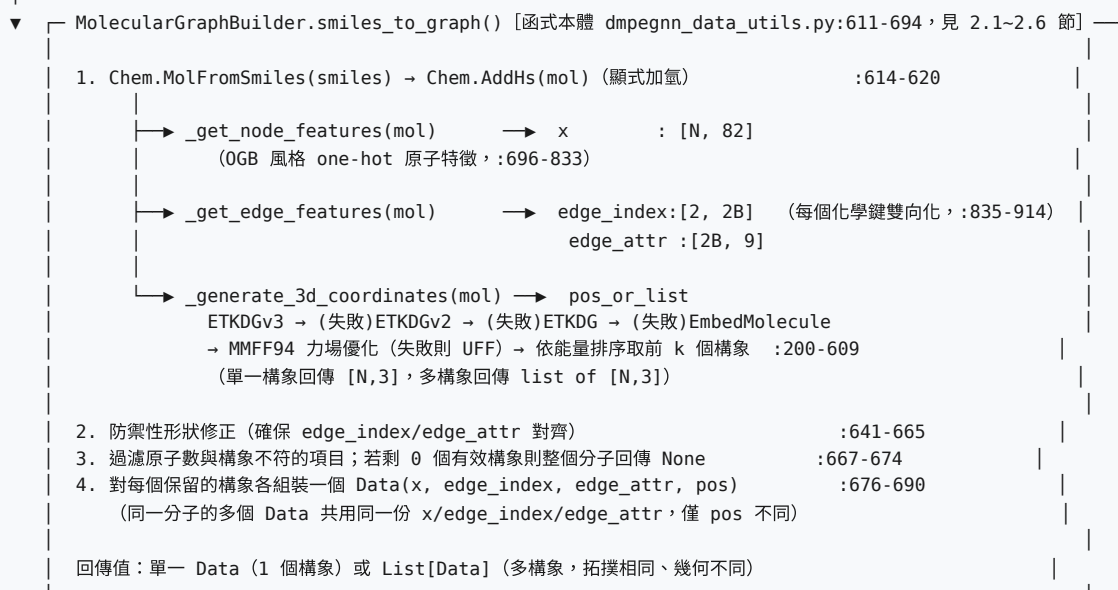
- **GAT 風格的加性注意力機制**（2D 拓撲分支）
- **Directed Message Passing (DMP)** —— D-MPNN/Chemprop 風格的有向邊訊息傳遞，具備「排除反向邊」的 no-backtracking 機制
- **EGNN 風格的等變 (equivariant) 3D 座標更新** —— 座標增量僅由不變純量（如 `dist_sq`）縮放相對位置向量而得，理論上保持 E(3) 等變性
- **geo_gate 幾何門控機制** —— 以聚合後的 3D 幾何訊息調變 2D 節點特徵，並使用 `bias=+2.0` 的冷啟動技巧

在完整融合模型 `DMPEGNN_MMB_Desc_Model`（`models.py:737-827`）中，DMP-EGNN 的圖級表示會與 MegaMolBART（256 維 SMILES 語言模型嵌入）、RDKit 正規化描述符（200 維）以簡單串接（concatenation）方式融合，經共享 MLP（`SharedMLP`）後由任務專屬線性頭（`task_heads`）輸出最終預測。

閱讀指引：本報告第 2~4 節依「SMILES 輸入 → 特徵工程 → DMP-EGNN 編碼 → 多模態融合 → 訓練/評估」的實際執行順序組織，每一項技術陳述均標註來源檔案與行號（如 `edmpnn_model_new.py:406-407`），可直接對照原始碼查核。第 6 節「優化建議」為分析後的獨立產出，依優先級（P1 高/P2 中/P3 低）排序，每項附具體理由與預期效益。

1. 整體資料流程總覽

SMILES 字串 (TDC ADMET 資料集, 例如 `herg / cyp3a4_substrate / half_life_obach`)



▼ `dmpgenn_dataset._build_dmpgenn_graphs()`
+ `descriptastorus RDKit2DNormalized()` → `descriptor: [200]` (分子層級, 非逐構象)
+ `y` (TDC 標籤) + `molecule_idx` (構象→分子映射)
+ 序列化快取至 `processed_tdc_data_dmpgenn/<dataset>/seed<n>/*.pt`

▼ `DMPEGNNGraphDataset.__getitem__(mol_idx)` → `List[Data]` (該分子所有構象)

▼ `collate_dmpgenn_multi()` → 展平為單一 PyG Batch
`batch.x / edge_index / edge_attr / pos / batch` (節點→圖)
`batch.molecule_idx, batch.smiles, batch.descriptor[Nmol,200], batch.y[Nmol]`

▼ `DMPEGNN_MMB_Desc_Model.forward()`

```
graph TD
    A[dmpegnn_backbone(x, edge_index, edge_attr, batch, pos, ...,  
return_graph_features=True, compute_logits=False)] --> B[→ AEGNNM.forward(): 見第 3 節完整計算流程  
→ graph_features (每個構象一筆)  
→ global_mean_pool(graph_features, molecule_idx) 聚合回「每分子一筆」]
    A --> C[mmb_model(smiles) → mmb_embeddings [Nmol, 256] (每次前向即時計算, 凍結骨幹)]
    C --> D[desc = batch.descriptor.view(-1, 200)]
    D --> E[combined = cat([graph_features, mmb_embeddings, desc], dim=1)  
shared = SharedMLP(combined) (3 層 Linear→Norm→ReLU→Dropout, 預設 hidden=128)  
logits = task_heads[0](shared) (單一線性層, task_index 目前恆為 0)]
```

▼ `train_utils.train/valid/test()`
`loss = L1Loss` (迴歸, 可選 `log1p` 變換) 或 `BCEWithLogitsLoss` (分類)
`metric = TDC Evaluator` (MAE / ROC-AUC ...)

圖例說明：上圖第一個方框以「┌ 函式名稱 … ─┐」框線標示**函式的作用域**——框內的 `Chem.MolFromSmiles` / `Chem.AddHs` / `_get_node_features()` / `_get_edge_features()` / `_generate_3d_coordinates()` 以及最後的 `Data(...)` 組裝，**全部都是 `MolecularGraphBuilder.smiles_to_graph()` 這一個函式內部依序執行的程式碼**，而非彼此獨立、各自輸出後再被 `smiles_to_graph()` 收尾的「下一個步驟」。換言之：`_get_node_features()` / `_get_edge_features()` 只負責產生「裸的」節點/邊特徵陣列（尚不是一張圖），`smiles_to_graph()` 則是呼叫它們並把結果連同 3D 座標一起包成 PyG Data 物件——這一步才真正構成模型所吃的「圖」。後續 `dmpgmn_dataset._build_dmpgmn_graphs()` 才是在 `smiles_to_graph()` 回傳之後、真正屬於下一個獨立階段的呼叫方。

2. 第一部分：資料前處理與特徵工程

對應檔案：`core/dmpegnn_data_utils.py` (1823 行, `MolecularGraphBuilder` / `MolecularDataset` / `DataPreprocessor`) 與 `core/dmpegnn_dataset.py` (284 行, `DMPEGNNGraphDataset` 與 `collate` 函式)。

2.1 SMILES → RDKit Mol → 3D 構象生成

入口為 `MolecularGraphBuilder.smiles_to_graph()` (`dmpegnn_data_utils.py:611-694`)：

- 解析：**`Chem.MolFromSmiles(smiles)`，失敗回傳 `None` (整個分子被捨棄):614-616。
- 加氫：**`Chem.AddHs(mol)` (`add_hydrogens=True` 為預設值)，之後所有原子/鍵特徵與 3D 座標均在「顯式氫」的分子圖上計算，而非重原子圖:619-620。
- 3D 構象嵌入層級退化鏈 (cascading fallback)：**

階段	方法	觸發條件
1	<code>AllChem.ETKDGv3()</code> (<code>useExpTorsionAnglePrefs=True</code> , <code>useBasicKnowledge=True</code> , <code>randomSeed=42</code> , <code>pruneRmsThresh=0.5</code>)	預設首選
2	<code>AllChem.ETKDGv2()</code> (同參數)	階段 1 拋出例外
3	<code>EmbedMultipleConfs(useExpTorsionAnglePrefs=True, useBasicKnowledge=True, maxAttempts=50)</code> (無版本參數)	階段 2 拋出例外
4	<code>EmbedMultipleConfs(randomSeed=42, maxAttempts=50)</code> (純預設)	階段 3 仍產生 0 個構象 ID
5	退回單一構象路徑 <code>_generate_single_conformer()</code> (內部同樣有 <code>ETKDGv3→v2→ETKDG→EmbedMolecule</code> 四層退化)	階段 4 仍失敗

`dmpegnn_data_utils.py:254-343` (多構象)，:517-609 (單構象)

- 力場優化與能量排序：**優先 `AllChem.MMFFOptimizeMoleculeConfs(maxIters=200)` (MMFF94，僅保留 `notConverged==0` 的構象)；MMFF 失敗則退回 `UFFOptimizeMoleculeConfs`；兩者皆失敗則不做能量篩選，直接取第一個嵌入的構象:476-515。能量由低到高排序，若 `num_conformers_to_keep>1`，取能量最低的前 k 個構象的座標。
- 逾時保護 (僅多進程路徑)：**每個分子預設 300 秒 `SIGALRM` 逾時；逾時後以「輕量模式」(`num_conformers=3`，120 秒逾時)重試；仍失敗則永久排除該分子。此機制僅在 Unix 系統有效，其他平台會靜默略過逾時保護:926-1043。

2.2 原子 (節點) 特徵向量 —— 共 82 維

`_get_node_features()` (:696-833)，維度於 :780-783 以 `assert/ValueError` 硬性檢查：

#	特徵	編碼方式	維度	備註
1	原子種類	47 種固定元素 one-hot + 1 個 Unknown 桶	48	<code>atom.GetAtomicNum()</code>
2	degree	one-hot, <code>min(degree, 10)</code>	11	
3	形式電荷	one-hot, 索引 = <code>clip(charge+5, 0, 10)</code>	11	範圍 -5 ~ +5
4	雜化狀態	one-hot, SP/SP2/SP3/SP3D/SP3D2	5	其餘狀態全 0
5	是否芳香	二元純量	1	
6	總氫數	one-hot, <code>min(num_h, 5)</code>	6	顯式加氫後此值多為 0

發現：建構子接受 `use_chirality`、`use_hydrogen_bonds`、`use_atomic_number` 等特徵開關參數（:52-59），但 `_get_node_features` / `_get_edge_features` 完全未依這些旗標分支——82/9 維方案是寫死的，**手性 (chirality/CIP) 資訊從未被讀取**（無任何 `GetChiralTag()` 呼叫）。對於藥物分子而言，立體異構通常直接影響 ADMET 性質（例如代謝、毒性），這是一項實質的特徵缺口，詳見第 6 節建議 P1-1。

2.3 鍵結（邊）特徵向量與有向邊建構——共 9 維

#	特徵	編碼方式	維度
1	鍵結類型	one-hot : SINGLE/DOUBLE/TRIPLE/AROMATIC (未知類型預設為 SINGLE)	4
2	立體化學	one-hot : NONE/ANY/Z/E	4
3	是否共軛	二元純量	1

`_get_edge_features()`（:835-914）對每個 RDKit 鍵 (i, j) 同時寫入 $[i, j]$ 與 $[j, i]$ 兩個方向，特徵向量在兩個方向上完全相同（無 begin/end 原子的不對稱編碼），因此 `edge_index` 形狀為 $[2, 2B]$ （ B = 鍵數）。

重要發現——反向邊索引 (b2revb) 並非資料層產物：特徵工程檔案中**完全沒有**計算 D-MPNN 慣用的 `b2revb` / reverse-edge 對照表（對 `dmpgnn_data_utils.py` 與 `dmpgnn_dataset.py` 搜尋 `revb` / `reverse_edge` 均為零筆）。雖然邊 $2k$ 與 $2k+1$ 因建構順序天然互為反向對，但這個關係從未被物化成張量隨 `Data` 物件序列化。**反向邊映射是在模型端**（`edmpnn_model_new.py`:410-443 的 `build_b2revb()`）於每次前向傳播動態重新計算，屬於重複運算（見第 6 節 P2-1）。

2.4 3D 座標 (pos) 與有效性檢查

- 座標直接取自 RDKit conformer 的 `conf.GetPositions()`（Å 單位），**全檔案未進行任何置中 (mean-subtraction) 或正規化**——與力場優化後的原始構象座標完全一致:317,324,330-580,592,599。
- 多構象分子會產生多個 `Data` 物件，彼此 `x` / `edge_index` / `edge_attr` 完全相同、僅 `pos` 不同:676。
- 有效性檢查分為兩層：(a) 特徵工程層僅做「原子數是否與構象一致」的防禦性過濾，以及下游 `pos_norm.sum() > 1e-6` 的寬鬆檢查（:963-965, 1310-1312, 1650-1657）；(b) 模型層的 `check_valid_positions()`（`edmpnn_model_new.py`:16-64）則更嚴格，同時檢查 `isfinite`、非零比例 $\geq 10\%$ 、平均範數 $\geq 1e-4$ ——但後者僅在模型前向傳播時被呼叫，資料集端從未套用，代表快取於磁碟的資料可能含有僅通過寬鬆檢查、但無法通過嚴格檢查的邊界案例，全部留給模型在訓練時逐批動態判斷並優雅降級（見 3.5 節）。

2.5 隨機旋轉資料增強

兩個特徵工程檔案中**完全沒有**旋轉增強邏輯（`rotat` / `augment` 關鍵字零命中）。旋轉增強是**模型前向傳播時的訓練期操作**（見 3.4 節），因此磁碟上快取的 `pos` 張量永遠是未增強的原始構象方向。

2.6 最終 PyG Data 物件與 DMPEGNNGraphDataset

欄位	形狀	來源
x	[N, 82]	MolecularGraphBuilder
edge_index	[2, 2B]	同上
edge_attr	[2B, 9]	同上
pos	[N, 3]	同上（每構象各一份）
descriptor	[200]	並非builder內建的~217維 Descriptors.CalcMolDescriptors，而是 dmpegnn_dataset.py:88-99 另外呼叫 descriptastorus.RDKit2DNormalized() 計算的獨立 200 維正規化描述符（分子層級，去除首個成功旗標元素）
y / smiles	[1] / str	逐構象複製同一分子標籤，供追溯

DMPEGNNGraphDataset（dmpegnn_dataset.py:19-46）以「分子」而非「構象」為索引單位：__len__ 回傳分子數，__getitem__(mol_idx) 回傳該分子所有構象的 List[Data]。collate_dmpegnn_multi()（:49-85）在批次整併階段才展平所有構象成單一 PyG Batch，並附加 molecule_idx / smiles / descriptor / y 等分子層級欄位。快取檔依 <dataset_name>/seed<n>/<split>_multi_keep<k>.pt 命名，快取鍵未納入 num_conformers、prune_rms_thresh、add_hydrogens 等 builder 設定的雜湊值，變更這些設定但沿用同一資料集名稱會靜默載入過期快取（見第 6 節 P2-3）。

3. 第二部分：DMP-EGNN 核心模型架構

對應檔案：`core/edmpnn_model_new.py`（1356 行）。模型類別在檔案內部仍稱為 `AEGNNM`，於 `models.py:10` 匯入時別名為 `DMPEGNN`。

3.1 模組組成與關鍵超參數

類別	行號	角色
<code>check_valid_positions</code> （函式）	16-64	座標張量嚴格有效性檢查
<code>DropPath</code>	67-81	殘差分支的 Stochastic Depth
<code>FocalLoss</code>	84-168	類別不平衡分類損失
<code>GATEGNNLayer(MessagePassing)</code>	171-573	核心層 ：GAT 注意力 + 有向訊息傳遞 + <code>geo_gate</code> + EGNN 座標更新
<code>GraphAttentionPooling</code>	576-645	已實作但 未被使用 的注意力池化（死代碼）
<code>AEGNNLayer</code>	648-757	包裝一個 <code>GATEGNNLayer</code> + FFN + 殘差/LayerNorm
<code>AEGNNM</code>	760-1117	主幹：嵌入層、 <code>num_layers</code> 層堆疊、多模態池化融合、輸出投影
<code>AEGNNMRegressor</code> / <code>AEGNNMClassifier</code>	1120-1309	任務特定損失（L1/Spearman；CE/Focal/BCE/ClassBalancedFocal）

預設超參數（`AEGNNM.__init__`，`:763-788`；經 Optuna 於 `train/optuna_train.py:122-144` 動態搜索）：

參數	預設值	Optuna 搜索範圍（依資料集大小分級）
<code>hidden_dim</code>	256	{64,128} / {128,256,384}
<code>num_layers</code>	6	[2, 3~6]（依資料量分級上限）
<code>num_heads</code>	8	{4,8}
<code>dropout</code>	0.1	[0.3, 0.5]（step 0.05）
<code>dmp_steps</code>	2	[1, 4]
<code>pool_type</code>	mean	{mean, sum}
<code>rotate_aug</code>	False	—（未列入搜索）
<code>use_coord_branch</code>	False	—（避免絕對座標洩漏，:950-952）

3.2 Directed Message Passing（DMP）核心機制

初始邊訊息（於 `_egnn_update` 內即時計算，`:343-408`，非資料建構期產物）：

```
rel_pos = pos[target] - pos[source] # :355
dist_sq = clamp((rel_pos**2).sum(-1), min=1e-8) # :356-359 （旋轉/平移不變純量）
h_i, h_j = h[target], h[source] # GAT 注意力輸出後的節點特徵
edge_feat = edge_attr_proj(edge_attr) # 可選
e_ij_0 = mlp_edge_init( concat[h_i, h_j, dist_sq, (edge_feat)] ) # Linear--SiLU-Linear
```

D-MPNN 風格「排除反向邊」更新規則（`_run_directed_mp`，`:508-562`，這是目前生效的實作；`:445-506` 為以三引號字串保留的舊版死代碼）：


```

for t in range(dmp_steps):
    incoming_sum[n] = scatter_add(e, target=n) for all nodes n # :548-549 對每個節點加總所有指向它的邊訊息
    neighbor_sum[i-j] = incoming_sum[source_i] - e[reverse(i-j)] # :555
    # 即  $\sum_{k \in N(i) \setminus \{j\}} e_{k-i}$  — 排除  $j-i$  訊息「原路傳回」 $i-j$ ，避免資訊迴圈
    delta = mlp_edge_update( concat[e_init, neighbor_sum] ) # Linear(2H-H)-SiLU-Linear(H-H)
    e = e_init + delta # :561 錨定於初始訊息的殘差式更新

```

DMP 步驟結束後，邊訊息聚合回節點：`node_messages[i] =  $\sum_j e_{ij}$` （:393-395），同時餵入節點更新 MLP `phi_h` 與 `geo_gate`。

3.3 GATEGNNLayer：注意力機制與 geo_gate

注意力機制為加性（additive）GAT 風格，並非點積注意力：

```

h = W(x).view(N, heads, head_dim) # :287, W 無 bias
e_raw = (concat[h_i, h_j] * att).sum(-1) #  $a^T [w_h_i || w_h_j]$ , :328-331
e_raw += (edge_attr_transformed * att_edge).sum(-1) # 可選邊注意力項, :334-336
alpha = softmax_per_target_node(e_raw, index=edge_index[target]) # PyG scatter-softmax, :339
out = scatter_add( x_j * alpha (+ edge_attr) , target ) # message()+propagate, :564-573

```

發現：建構子存有 `alpha=0.2`（LeakyReLU 斜率）參數並一路傳遞至 `AEGNNM`，但 `_compute_attention()` 從未對注意力分數套用 LeakyReLU 或任何非線性——這是原版 GAT 論文中的標準作法（防止注意力 logits 在 softmax 前退化為近似線性），目前該參數是**形同虛設的殘留超參數**（見第 6 節 P2-2）。

`geo_gate` 幾何門控機制（:240-254, 401-407）——本模型的核心創新點：

```

phi_h_input = concat[h, node_messages] # [N, 2*hidden], h=GAT分支輸出, node_messages=DMP聚合後的3D訊息
h_candidate = phi_h(phi_h_input) # Linear(2H-H)-SiLU-Linear(H-H) 標準EGNN節點更新
geo_gate = Sigmoid( Linear(H-H)( Dropout( SiLU( Linear(2H-H)(phi_h_input) ) ) ) )
h_new = h_candidate * geo_gate # 逐元素相乘：以幾何上下文調變2D特徵

```

設計亮點——冷啟動偏置初始化：`nn.init.constant_(geo_gate[-2].bias, 2.0)`（:271）使 `sigmoid(2.0)≈0.88`，讓 gate 在訓練初期「近乎全開」，避免 `phi_h` 輸出在權重尚未學習前就被預設零偏置（`sigmoid(0)=0.5`）腰斬一半，是穩定訓練初期梯度流的巧妙作法，且因 gate 僅由不變純量／特徵驅動（非原始座標），仍保持等變性。

座標（equivariant position）更新（:374-391）：`coord_coeff = 1/deg(i)`（度數正規化）、`phi_x_val = tanh(phi_x(e_ij))`（有界純量）、再乘上一個獨立的 sigmoid `coord_gate`（穩定更新幅度），最終 `pos += scatter_add(coord_coeff * rel_pos * phi_x_val * coord_gate)`——僅以不變純量縮放相對位置向量，符合標準 EGNN 等變設計。**座標會逐層更新並向下傳遞**（非僅用於算一次距離特徵），每層都重新計算 `dist_sq` 餵給該層的 gate。

3.4 隨機旋轉資料增強（模型內）

訓練期、且 `rotate_aug=True`（預設為 `False`，需手動開啟）時觸發：`pos = _apply_random_rotation(pos, batch)`（:915-916）。旋轉矩陣以 **Rodrigues 旋轉公式**生成（非 QR 分解、非歐拉角）：隨機單位軸 + 隨機角度 → 反對稱矩陣 $K \rightarrow R = I + \sin(\theta)K + (1-\cos(\theta))K^2$ ，透過 `torch.bmm` 對批次內每個分子各自生成獨立旋轉矩陣並向量化套用（:1046-1103）。

3.5 check_valid_positions() 與優雅降級

檢查條件：(1) `isfinite` 全通過；(2) 非零座標比例 $\geq 10\%$ ；(3) 平均範數 $\geq 1e-4$ （:16-64）。**未檢查共線性或退化幾何**（三點共線、原子重疊）。若驗證失敗，**不拋例外、不記錄警告**——`GATEGNNLayer.forward`（:310-319）直接跳過整個 `_egnn_update`，該層靜默退化為純 2D GAT 注意力層，`pos` 原樣透傳。此為穩健性設計，但同時也意味著 3D 幾何失效時訓練日誌中不會有任何線索（見第 6 節 P2-4）。

3.6 池化（Readout）

發生在本檔案內（非 `models.py`）：節點特徵依 `pool_type`（mean/sum/norm）池化為 `node_mean`，經 `h_mlp`（`hidden-hidden/2`）得到 `h_processed`；再視 `use_coord_branch` / `use_fingerprint` / `use_descriptor` 旗標決定是否串接座標分支、指紋分支、描述符分支，最終 `graph_features = concat(active_branches)`（:932-1001）。**GraphAttentionPooling** (:576-645) 已完整實作但從未被 `forward()` 呼叫，目前池化僅用 PyG 內建的 `global_mean_pool` / `global_add_pool`。

3.7 完整前向傳播偽代碼

```
AEGNNM.forward(x, edge_index, edge_attr, batch, pos, fingerprint, descriptor, b2revb,
               return_graph_features, compute_logits):

    x = node_embedding(x)                # Linear(82 → hidden_dim)
    edge_attr = edge_embedding(edge_attr) # Linear(9 → hidden_dim)
    if rotate_aug and training and pos is not None:
        pos = _apply_random_rotation(pos, batch) # §3.4

    for layer in aegnn_layers:           # xnum_layers (預設6)
        residual = x
        gat_out, attn, updated_pos = GATEGNNLayer(x, edge_index, edge_attr, pos, b2revb)
        # ↳ 內部：GAT注意力(§3.3) → propagate → 若 pos 有效(§3.5)：
        #     _egnn_update：DMP(§3.2, dmp_steps次) → 座標更新 → geo_gate(§3.3) → h_new
        x = norm1(residual + drop_path(gat_out)) # post-norm (預設)
        x = norm2(x + drop_path(ffn(x)))         # FFN: Linear-SiLU-Dropout-Linear
        if updated_pos is not None: pos = updated_pos # 座標逐層傳遞

    node_mean = pool(x, batch, pool_type) # mean/sum/norm
    h_processed = h_mlp(node_mean)        # hidden → hidden/2
    feature_list = [h_processed]
    if use_coord_branch: feature_list.append(x_mlp(pool(pos, batch)))
    if use_fingerprint:  feature_list.append(fingerprint_mlp(fingerprint) [* gate])
    if use_descriptor:   feature_list.append(descriptor_mlp(descriptor))
    graph_features = concat(feature_list)

    if compute_logits:
        logits = output_proj(nan_to_num(graph_features)) # 純 Linear，無 norm
    else:
        logits = zeros(batch_size, output_dim) # 融合模型呼叫時傳 compute_logits=False

    return logits, attention_weights[, graph_features]
```

4. 第三部分：多模態融合與訓練流程

對應檔案：`core/models.py` (979 行)、`core/model_factory.py` (222 行)、`core/train_utils.py` (226 行)。

4.1 融合模型家族與 DMPEGNN 在其中的位置

`models.py` 定義了一系列「編碼器 × SharedMLP × task_heads」的融合模型，涵蓋 GCN、Chemprop MPN、DMPEGNN、AEGNN（舊版）等骨幹與 MegaMolBART（MMB）、RDKit 描述符（Desc）的排列組合。與 DMP-EGNN 相關者：

類別	行號	融合模態
<code>DMPEGNN_Fusion_Model</code>	577-669	僅 DMP-EGNN 圖嵌入
<code>DMPEGNN_Desc_Model</code>	672-734	DMP-EGNN + RDKit 描述符
<code>DMPEGNN_MMB_Desc_Model</code>	737-827	DMP-EGNN + MegaMolBART + RDKit 描述符（三模態融合，主力模型）

三個類別均呼叫骨幹 `forward(..., return_graph_features=True, compute_logits=False)` ——即完全不使用 DMP-EGNN 自帶的輸出投影頭，只取池化後的 `graph_features`，於外部重新融合。骨幹內部的 `use_fingerprint=False`, `use_descriptor=False`, `use_coord_branch=False` (:612-617)，故 `graph_repr_dim = hidden_dim/2`（僅 H 分支）。

4.2 融合機制：串接 + 共享 MLP（無跨模態注意力）

```
# DMPEGNN_MMB_Desc_Model.forward()                                models.py:737-827
graph_features = dmpegnn_backbone(x, edge_index, edge_attr, batch, pos, ...) # 每個「構象」一筆
graph_features = global_mean_pool(graph_features, molecule_idx)             # 聚合回「每分子」一筆      :816-820
mmb_embeddings = mmb_model(smiles)                                         # 每次前向即時計算（凍結骨幹） :821
desc = descriptor.view(-1, 200).float()                                    # 預先計算並快取於磁碟      :824
combined = concat([graph_features, mmb_embeddings, desc], dim=1)           # 128 + 256 + 200 = 584 維（預設）
shared = SharedMLP(combined)                                              # 3層
Linear_Norm_ReLU_Dropout(0.2), hidden=128
logits = task_heads[0](shared)                                           # 單一 Linear(128-1)
```

效率發現：MegaMolBART 分支的骨幹權重已凍結（`train/seed_train.py:136-137`），輸出對相同 SMILES 恆定不變，但**每個訓練 step 都重新對整批 SMILES 執行一次 NeMo Transformer 編碼**（`models.py:821`），而 RDKit 描述符卻在資料建構期算好並快取到磁碟（`dmpegnn_dataset.py:88-99`）。兩者處理方式不對稱，MMB 分支存在明顯可消除的重複運算（見第 6 節 P1-2）。

4.3 Task Heads 與單任務現況

`task_heads = ModuleList([Linear(shared_dim, out_dim) for out_dim in task_output_dims])` (:634-636 等) ——每個任務僅一個獨立線性層，共享主幹完全相同、不含任何任務條件化（task-conditioning）機制。訓練管線目前恆常使用 `task_index=0`（`train_utils.py:165,173,181,194,202`；`optuna_train.py:69` 預設 `num_tasks=1`）——多任務結構已存在但從未真正啟用，每次訓練僅針對單一 ADMET 屬性（見第 6 節 P1-3）。

4.4 model_factory.get_model() 建構流程

依 `model_type` 字串分派（支援 GCN/MMB/DESC/GCN_MMB/MMB_DESC/GCN_DESC/GCN_MMB_DESC/MPN*/DMPEGNN/DMPEGNN_DESC/DMPEGNN_MMB_DESC/AEGNN/AEGNN_DESC）。`DMPEGNN_MMB_DESC` 分支（:159-181）以 `args.dmpegnn_hidden_dim/num_layers/num_heads/dropout/dmp_steps/pool_type` 建構原始 AEGNNM 骨幹（`node_features=82`, `edge_features=9` 寫死，對應第 2 節的特徵工程輸出），再包入 `DMPEGNN_MMB_Desc_Model`，並注入外部預先建好的 `megamolbart_model`。

4.5 訓練 / 驗證 / 測試迴圈（train_utils.py）

函式	行為
<code>train()</code>	跳過 <code>batch.batch.size(0)<2</code> 的批次；梯度裁剪 <code>max_norm=1.0</code> ； <code>avg_loss = total_loss / num_batches</code> （分母正確，僅計已處理批次）
<code>valid()</code>	同樣跳過小批次，但 <code>avg_loss = total_loss / len(loader)</code> （分母錯誤——用 DataLoader 總批次數而非實際處理數，被跳過的批次仍計入分母，導致 loss 系統性低估）；分類套用 <code>sigmoid</code> ；若 <code>log_transform</code> ，於 loss 計算後對 <code>pred/label</code> 做 <code>expm1</code> 還原原始尺度
<code>test()</code>	僅推論，無 loss；分類 <code>sigmoid</code> +轉整數標籤，迴歸可選 <code>expm1</code>
<code>model_forward()</code>	依 <code>model_type</code> 的字串分派表，將 PyG Batch 欄位解包成各融合模型的 <code>forward()</code> 簽名； <code>DMPEGNN_MMB_DESC</code> 分支固定傳入 <code>task_index=0</code>

4.6 端到端流程（以 DMPEGNN_MMB_DESC 為例）

1. TDC `admet_group.get(dataset_name)` 取得 train/valid/test 的 SMILES + 標籤。
2. `_build_dmpegnn_graphs()` 一次性特徵工程並快取（第 2 節），失敗分子（3D 生成失敗、描述符合 NaN/全零）被排除。
3. 可選 `log1p` 標籤變換（`seed_train.py:38-53`，就地修改 dataset，需注意物件複用風險，見第 5 節）。
4. `DMPEGNNGraphDataset` + `collate_dmpegnn_multi` 組成 `DataLoader`。
5. `model_factory.get_model()` 建構 `DMPEGNN_MMB_Desc_Model`（含凍結的 MegaMolBART）。
6. 逐 epoch： `train()` → `valid()`（含 TDC Evaluator 計算 ROC-AUC/MAE）→ early stopping / 最佳權重存檔。
7. 最終於 test set 呼叫 `test()`，回報測試分數與預測值。

5. 已知程式碼問題彙整表

以下彙整本次分析（及先前 `code_review_dmpegnn.md` 審查報告）發現、且截至分析當下仍存在於程式碼中的問題，依嚴重度排列。

嚴重度	檔案:行號	問題	影響
HIGH	<code>train_utils.py:75</code>	<code>valid()</code> 的 <code>avg_loss = total_loss / len(loader)</code> 未排除被跳過的小批次	驗證 loss 系統性低估，影響 Optuna 超參搜索與 early stopping 判斷（ <code>train()</code> 已修正， <code>valid()</code> 未同步修正）
HIGH	<code>dmpegnn_dataset.py:35</code>	<code>num_molecules = max(molecule_indices) + 1</code> 對空列表無防護	當某個 split 所有分子均特徵化失敗時，程式以難以理解的 <code>ValueError: max() arg is an empty sequence</code> 崩潰
MEDIUM	<code>dmpegnn_data_utils.py:52-59, 696-914</code>	<code>use_chirality / use_hydrogen_bonds</code> 等特徵開關被接受但從未實際生效	手性資訊完全遺失；配置參數造成錯誤的可配置性假象
MEDIUM	<code>edmpnn_model_new.py:185, 321-341</code>	<code>alpha</code> （LeakyReLU 斜率）宣告但從未在注意力計算中使用	殘留死參數；注意力 logits 缺少非線性可能略微限制表達力
MEDIUM	<code>models.py:821</code>	MegaMolBART 嵌入每個 step 即時重算，未快取	顯著的可避免訓練時間開銷（骨幹已凍結，輸出對同一 SMILES 恆定）
MEDIUM	<code>dmpegnn_dataset.py:118-119</code>	磁碟快取鍵未納入 <code>num_conformers / prune_rms_thresh / add_hydrogens</code> 等 builder 設定	修改特徵工程設定但沿用相同 <code>dataset_name</code> 會靜默載入過期快取
LOW	<code>edmpnn_model_new.py:445-506</code>	舊版 <code>_run_directed_mp</code> 以三引號字串保留為死代碼	可讀性、linter 誤判 docstring 風險
LOW	<code>edmpnn_model_new.py:576-645</code>	<code>GraphAttentionPooling</code> 已實作但未被使用	死代碼；亦是唾手可得的池化升級選項（見 P2-6）
LOW	<code>seed_train.py:39-54</code>	<code>_apply_log1p_to_dataset</code> 就地修改傳入的 dataset，無副作用標注	若呼叫方複用同一 dataset 物件， <code>log1p</code> 可能被重複疊加

6. 架構與演算法優化建議（依優先級排序）

以下建議基於第 2~5 節的逐行分析歸納，依 **P1 高** / **P2 中** / **P3 低** 優先級排序。P1 為預期投資報酬率最高、風險/成本比最低的項目。

P1 資料與特徵層面

P1-1 補上手性（chirality）特徵

問題：82 維原子特徵完全未編碼 `atom.GetChiralTag()` / CIP 標籤，`use_chirality` 旗標形同虛設（`dmpegnn_data_utils.py:696-833`）。**理由：**許多 ADMET 終點（代謝穩定性、CYP 受質選擇性、hERG 結合）對立體異構高度敏感——同一 SMILES 骨架的對映異構體可能有截然不同的活性。目前模型連 3D 座標都納入了，卻在拓撲特徵層遺漏這個低成本、高資訊量的訊號。**建議：**在 `_get_node_features` 增加 4~5 維 one-hot（CHI_UNSPECIFIED/CW/CCW/OTHER），並真正依 `use_chirality` 旗標控制，同時同步更新 `node_feature_dim` 與 `model_factory.py` 中寫死的 `node_features=82`。**成本：**低（純特徵工程改動，不影響模型結構）。

P1-2 快取 MegaMolBART 嵌入，避免逐 step 重算

問題：`models.py:821` 每個訓練 step 都對整批 SMILES 重新執行凍結的 NeMo 編碼器前向傳播。**理由：**骨幹權重已凍結（`seed_train.py:136-137`），對同一 SMILES 輸出恆定；RDKit 描述符已示範了「資料建構期算好、序列化快取」的正確模式（`dmpegnn_dataset.py:88-99`），MMB 嵌入應採用相同策略。以典型 Optuna 50 trials × 5 seeds × 數十 epoch 的訓練量，重複計算的成本相當可觀，尤其在 GPU 資源有限時直接影響超參搜索的週轉時間。**建議：**在 `_build_dmpegnn_graphs`（或獨立的預處理步驟）中，對每個唯一 SMILES 呼叫一次 `mmb_model(smiles)` 並將 256 維向量與 `descriptor` 一併存入 `Data` / 快取檔；訓練時直接讀取，僅在 `head`（LayerNorm+Linear）需要梯度時才需要區分「凍結部分預先算」與「可訓練 head 每次算」——若 `head` 本身很小，也可以只快取 512 維的 pooled 編碼器輸出（`head` 之前），讓 `head` 仍可訓練。**成本：**中（需調整資料管線與快取格式，但 SharedMLP/task_heads 介面不變）。

P1-3 啟用真正的多任務訓練

問題：`task_heads` / `task_index` 架構完整存在，但訓練管線恆用 `task_index=0`、`num_tasks=1`（`train_utils.py` 多處、`optuna_train.py:69`），實際上是「每個 ADMET 屬性訓練一個獨立模型」。**理由：**ADMET 屬性間存在已知的化學/生物關聯（如多個 CYP 同功酶受質性、多個毒性終點共享結構驅動因子），多任務學習在藥物性質預測文獻中已被證實能顯著提升低資料量終點（如 `half_life_obach` 等回歸任務）的資料效率與泛化能力，且不需改動模型骨幹——只需要在同一個共享 trunk 上同時訓練多個 `task_head`。**建議：**優先在同屬性類型（如同為分類的 CYP 系列，或同為迴歸的 PK 參數）之間試點聯合訓練，訓練迴圈需改為依批次的任務標籤選擇對應 `task_index` 並加權合併多任務損失。**成本：**中高（需要重新設計 DataLoader 的跨資料集批次組織方式與損失加權策略）。

P2 模型架構層面

P2-1 預先計算並快取 b2revb 反向邊索引

問題：`build_b2revb()`（`edmpnn_model_new.py:410-443`）以 Python 字典雜湊在每次前向傳播動態重建反向邊映射。**理由：**對固定分子拓撲而言 `edge_index` 不隨訓練改變，`b2revb` 是純拓撲函數，重複計算是可避免的 CPU 開銷（尤其在大 batch、GPU 前向很快時容易成為 CPU-bound 瓶頸）。**建議：**在特徵工程階段（`dmpegnn_data_utils.py` 建構 `edge_index` 的同一函式中）直接以「偶數邊 2k 對應奇數邊 2k+1」的建構順序性質，一次性計算 `revedge_index = torch.arange(2B).view(-1,2).flip(1).flatten()` 並存為 `Data.revedge_index`，隨圖一併序列化快取，模型 `forward()` 改為優先使用傳入的 `b2revb`（該參數已存在但目前訓練程式碼從未傳入，見 `train_utils.py` 的呼叫）。**成本：**低（資料端一行改動+模型端把 `None` 預設改為使用傳入值）。

P2-2 為注意力 logits 補上 LeakyReLU 或移除死參數

問題：`alpha=0.2` 一路傳遞卻從未用於注意力計算（`:321-341`）。**建議：**二擇一——(a) 依原始 GAT 設計在 `e_raw` 上套用 `F.leaky_relu(e_raw, negative_slope=alpha)` 後再做 softmax（低成本 A/B 測試即可驗證是否提升驗證分數）；(b) 若消融後無明顯差異，直接移除該參數鏈以減少配置面混淆。**成本：**極低。

P2-3 快取鍵納入特徵工程設定雜湊

問題：見第 5 節彙整表。**建議：**在 `_build_dmpegnn_graphs` 的快取檔名中加入 `hashlib.md5(str(sorted(builder_config.items())).encode()).hexdigest()[:8]` 之類的短雜湊，變更任一 builder 參數即自動失效舊快取，避免「改了設定卻用到舊資料」的隱性 bug。**成本：**極低。

P2-4 為 check_valid_positions 失敗路徑加上可觀測性

問題：3D 幾何驗證失敗時模型靜默退化為純 2D GAT，訓練日誌無任何線索（`edmpnn_model_new.py:310-319`）。**理由：**若某資料集因 3D 生成品質不佳導致大比例批次退化為 2D-only，使用者完全無法從訓練曲線察覺「3D 分支其實幾乎沒在起作用」，容易誤判模型架構本身的貢獻。**建議：**維護一個 per-epoch 計數器（如透過 `nn.Module.register_buffer` 或簡單屬性）記錄 `_egnn_update` 被跳過的節點/批次比例，並在訓練腳本中定期記錄（log）此統計量，作為資料品質與模型行為的診斷指標，而非改變運行時邏輯本身。**成本：**低。

P2-5 以 Boltzmann 加權取代簡單平均聚合多構象

問題：多構象生成過程中已計算並依 MMFF/UFF 能量排序（`dmpegnn_data_utils.py:476-515`），但這些能量值在保留 top-k 構象後即被完全丟棄，未隨 Data 物件傳遞；下游 `global_mean_pool(graph_features, molecule_idx)`（`models.py:816-820`）對所有保留構象一視同仁地簡單平均。**理由：**3D 分子性質預測文獻中常見作法是以 Boltzmann 權重 $w_i \propto \exp(-E_i/kT)$ 對多構象表示加權平均，賦予低能量（更可能實際存在）構象更高權重，這是符合物理直覺且僅需額外傳遞一個標量的低成本改進。**建議：**在 `smiles_to_graph()` 回傳的 Data 中新增 `conformer_energy` 欄位，並在融合模型的多構象聚合步驟改用能量加權的 `scatter_add`（或至少提供加權平均作為 `pool_type` 的新選項）取代等權重 `global_mean_pool`。**成本：**低中（資料端需傳遞能量；模型端聚合邏輯需小改）。

P2-6 將池化升級為已實作但閒置的 GraphAttentionPooling

問題：`GraphAttentionPooling`（`edmpnn_model_new.py:576-645`）已完整實作卻從未接入 `forward()`。**理由：**圖級任務中注意力池化通常優於均值/加總池化，因為它能让模型自行學習哪些原子（如藥效基團、反應中心）對分子級性質更關鍵，而非平等對待所有原子（包含大量顯式氫原子，因加氫後 H 原子占節點總數相當比例）。**建議：**將其接入 `pool_type='attention'` 作為 Optuna 可搜索的新選項，與現有 mean/sum/norm 並列比較，成本低且已有現成程式碼，是「唾手可得的改進」。**成本：**低（連接既有程式碼，非新增邏輯）。

P3 進階架構方向（需較多實驗驗證）

P3-1 引入非鍵結（non-bonded）空間邊，補足長程 3D 幾何資訊

問題：目前 3D EGNN 分支的邊集合完全等於化學鍵集合（`edge_index` 僅由 `mol.GetBonds()` 雙向化而來，`dmpegnn_data_utils.py:849-898`），代表座標更新與 `dist_sq` 幾何特徵只能感知「經由化學鍵相連的原子對」之間的相對位置，從未捕捉分子摺疊後在空間上鄰近、但拓撲上相隔多鍵的非共價接觸（分子內氫鍵、疏水口袋幾何、環系統的空間堆疊）。**理由：**對 hERG 阻斷、膜穿透性、蛋白結合等高度依賴分子 3D 形狀與非共價交互作用的 ADMET 終點而言，這類「through-space」而非「through-bond」的幾何資訊可能是重要但目前結構性缺失的訊號，這正是 SchNet/DimeNet 等純 3D 分子模型採用半徑圖（radius graph）而非僅用化學鍵圖的原因。**建議：**在特徵工程階段，以固定截止半徑（如 4~5 Å）或 k-近鄰額外建立「空間邊」，以獨立的邊類型旗標區分「鍵結邊」與「空間邊」餵入 DMP，可先做小規模消融實驗（挑 1~2 個對 3D 形狀敏感的資料集如 herg）驗證是否有效再全面推廣。**成本：**中高（涉及圖結構改動、需重新產生所有快取、模型需區分邊類型）。

P3-2 以徑向基底函數（RBF）展開距離，取代原始 dist_sq 純量

問題：`_egnn_update` 中距離資訊僅以單一純量 `dist_sq` 餵入 `mlp_edge_init`（`:356-369`）。**理由：**單一純量難以讓下游 MLP 學到距離的非單調（non-monotonic）效應（例如凡得瓦接觸在特定距離區間有吸引/排斥轉折），SchNet 系列模型改用高斯 RBF 展開 $\{\exp(-(d-\mu_k)^2/\sigma^2)\}$ 大幅提升了距離敏感任務的表現，屬社群已驗證的低風險改動。**建議：**將 `dist_sq`（或其平方根 `dist`）以 16~32 個高斯基底展開後再輸入 `mlp_edge_init`，維度增加可由 MLP 第一層吸收。**成本：**低中（純模型內部改動，不影響資料管線）。

P3-3 以跨模態注意力取代三模態簡單串接

問題：DMP-EGNN/MMB/Desc 三種異質模態目前僅以 `torch.cat` 串接後餵入單一 SharedMLP（`models.py:825-826`），三者被平等視為同一向量空間的片段，模型需自行從頭學習跨模態的相對重要性與交互作用。**建議：**可實驗以輕量的 cross-attention 或 gated fusion（如各模態先各自過一個 gate MLP 決定其對最終表示的貢獻權重，類似 `geo_gate` 的設計哲學延伸到模態層級）取代簡單串接，這與現有骨幹已展現的「用門控整合異質訊號」的設計語言一致，屬於漸進式而非顛覆性改動，建議先以小規模資料集驗證是否優於串接基線，避免過度工程化。**成本：**中（新增融合模組，SharedMLP 輸入介面需調整）。

優先執行順序建議

建議依「成本低、風險低、可獨立驗證」原則，優先完成 **P1-1（手性特徵）→ P2-2/P2-3/P2-6（低成本修正與死代碼啟用）→ P2-1（b2revb 快取）→ P1-2（MMB 快取，需搭配資料管線改版）→ P2-5（構象能量加權）**，待這批低風險改動驗證有效後，再評估是否投入 P1-3（多任務）與 P3 系列（結構性改動）等成本較高、需要更完整消融實驗設計的方向。同時強烈建議先修復第 5 節列出的兩個 HIGH 級別 bug（`valid()` 分母、空 `molecule_indices` 崩潰），因為它們會直接干擾任何後續優化的評估可信度——在 loss 計算本身有偏差的情況下，難以準確判斷架構改動是否真正帶來提升。

7. 結論

DMP-EGNN (AEGNNM) 是一個設計精巧的多分支 3D 分子圖神經網路，結合了 GAT 注意力、D-MPNN 式有向訊息傳遞與 EGNN 式等變座標更新，並以 `geo_gate` 機制優雅地將幾何訊息注入 2D 特徵、輔以合理的冷啟動偏置初始化。在多模態融合層，架構選擇了簡單但穩健的「串接 + 共享 MLP」策略，並透過動態讀取 `backbone.graph_repr_dim` 等實踐避免了維度硬編碼的脆弱性。

本次逐行分析同時發現若干具體、可操作的改進機會：從資料層面遺漏的手性特徵、模型層面閒置的 dead code (`alpha` 參數、`GraphAttentionPooling`) 與可避免的重複運算 (`b2revb`、MMB 嵌入)，到架構層面值得探索的方向 (非鍵結空間邊、RBF 距離展開、跨模態注意力)。這些建議已依成本與預期效益排序，建議團隊優先處理 P1/P2 中風險低、可獨立驗證的項目，並在修復兩個已識別的 HIGH 級別評估 bug 後，再系統性地評估更具實驗性的架構改動。